

PART II - VERSION 2 INDUSTRIAL PLC PROJECTS

Project 1 - Industrial Motor Control System

Version 2 - Industrial PLC Upgrade

PLC Platform: Allen-Bradley MicroLogix 1100

Programming Software: RSLogix Micro Starter Lite 8.30

Prepared By: Sharmin Rahman

1. Project Overview

Project 1 Version 2 upgrades the original motor start-delay project into a more structured PLC motor-control application. I retained the basic objective of delaying motor startup, but expanded the program to include operating-mode management, centralized start permissives, a system-ready status, fault handling, motor feedback monitoring, operator indication, start-event counting, and runtime accumulation.

The final program separates the operator's request to operate the motor from the physical Motor_Contactor output. Auto operation uses a latched Auto_Run_Request B3:0/1, while Manual operation uses the non-latched Mode_Run_Request B3:0/2. These requests are combined into Start_Request B3:0/3 and passed through Start_Permissive B3:0/0 before Start_Delay_Timer T4:0 can complete its five-second delay. Run_Command B3:1/0 then controls Motor_Contactor O:0/0 through a dedicated output rung.

Before startup is permitted, the PLC checks EStop_OK I:1/2, Stop_OK I:1/1, Overload_OK I:1/3, the latched Fault_Active B3:0/4 condition, and the operating-mode selection. Auto_Mode I:1/4 and Manual_Mode I:1/5 are also monitored by Mode_Fault B3:1/2 so simultaneous mode selection is treated as an invalid state.

Motor response is monitored after Motor_Contactor energizes. Feedback_Timer T4:1 provides a three-second validation interval. After that interval, the program checks Motor_Feedback I:1/8; missing feedback makes Feedback_Fault B3:0/7 true, which in turn latches the master Fault_Active condition. Fault_Active drives Fault_Light O:0/2 and Alarm_Horn O:0/3 and removes the internal Run_Command.

The program also records operating information. Rung 0015 uses T4:0/DN, ONS B3:2/0, and CTU C5:0 to record one count for each completed startup-delay event. Runtime_Timer T4:2 measures ten-second simulation intervals while Motor_Contactor is energized. Each completed interval uses ONS B3:2/1 and an ADD instruction to increment N7:0, after which T4:2 is reset to begin the next interval.

This Version 2 project demonstrates the progression from a simple timer-controlled motor exercise to a PLC program with clearer separation between operating requests, permissives, internal commands, physical outputs, feedback, diagnostics, and equipment-use information. The project was developed and tested in a PLC training/simulation environment. The standard PLC monitoring of E-stop and overload status shown here is not presented as a complete safety-rated machine control system.

2. Engineering Objective

The objective of Project 1 Version 2 was to redesign the original motor start-delay project using a more structured control architecture while preserving the core delayed-start function established in Version 1. I wanted the motor output to be the final result of validated operating conditions rather than a direct response to a Start input.

- Support separate Auto and Manual operating modes.
- Detect simultaneous Auto and Manual selection as an invalid mode condition.
- Prevent startup unless the implemented Stop, E-stop, overload, master-fault, and mode conditions permit operation.
- Create a clear System_Ready status before an operating request can be accepted.
- Maintain an automatic run request after a momentary Start command.
- Provide non-latched Manual run behavior through a dedicated Manual Run push button.
- Apply a five-second startup delay before energizing the motor contactor.
- Separate the internal Run_Command from the physical Motor_Contactor output.
- Monitor motor feedback after the contactor command is issued.
- Latch a master fault until the active fault conditions are restored and Reset is accepted.

- Provide visual and audible fault indication.
- Count completed startup-delay events and accumulate motor runtime intervals.

3. Functional Requirements

- The PLC shall monitor Start_PB I:1/0, Stop_OK I:1/1, EStop_OK I:1/2, Overload_OK I:1/3, Auto_Mode I:1/4, Manual_Mode I:1/5, Manual_Run_PB I:1/6, Reset_PB I:1/7, and Motor_Feedback I:1/8.
- Simultaneous Auto_Mode and Manual_Mode selection shall energize Mode_Fault B3:1/2.
- Mode_Fault, an unhealthy EStop_OK input, an unhealthy Overload_OK input, or Feedback_Fault B3:0/7 shall be able to latch Fault_Active B3:0/4.
- Start_Permissive B3:0/0 shall require EStop_OK, Stop_OK, Overload_OK, no active master fault, and exactly one valid operating mode.
- System_Ready B3:0/5 shall follow Start_Permissive.
- In Auto mode, Start_PB and System_Ready shall latch Auto_Run_Request B3:0/1.
- Auto_Run_Request shall unlatch if Stop_OK becomes false or Fault_Active becomes true.
- In Manual mode, Manual_Run_PB and System_Ready shall energize Mode_Run_Request B3:0/2 only while the enabling conditions remain true.
- Auto_Run_Request or Mode_Run_Request shall energize Start_Request B3:0/3.
- Start_Request and Start_Permissive shall enable Start_Delay_Timer T4:0 with a 1.0-second time base and preset of 5.
- When T4:0/DN is true and Fault_Active is false, Run_Command B3:1/0 shall energize.
- Run_Command shall energize Motor_Contactor O:0/0, and Motor_Contactor shall energize Running_Light O:0/1.
- Motor_Contactor shall enable Feedback_Timer T4:1 with a 1.0-second time base and preset of 3.
- When T4:1/DN is true and Motor_Feedback I:1/8 is false, Feedback_Fault B3:0/7 shall become true.
- T4:0/DN shall use ONS B3:2/0 before CTU C5:0 so the counter increments once per completed startup-delay event.
- Motor_Contactor shall enable Runtime_Timer T4:2 with a 1.0-second time base and a simulation preset of 10.
- Each T4:2/DN event shall pass through ONS B3:2/1 and execute $ADD\ N7:0 + 1\ to\ N7:0$.
- T4:2 shall be reset after each completed runtime interval.
- Fault_Active shall energize Fault_Light O:0/2 and Alarm_Horn O:0/3.
- Reset_PB shall unlatch Fault_Active only when EStop_OK and Overload_OK are healthy and both Mode_Fault and Feedback_Fault are false.

4. System Components

Component	Identification	Function
PLC Platform	Allen-Bradley MicroLogix 1100	Executes motor-control, fault, feedback, and runtime logic
Input Module	1762-IQ16	Provides discrete operator and monitoring inputs
Output Module	Built-in / configured discrete outputs	Provides motor, running, fault, and horn commands
Start Push Button	I:1/0	Initiates Auto start request
Stop Healthy Input	I:1/1	Normal stop-circuit status and Auto request unlatch condition
E-stop Healthy Input	I:1/2	Monitored E-stop circuit healthy status
Overload Healthy Input	I:1/3	Monitored motor overload healthy status
Auto Mode Selector	I:1/4	Selects automatic operation
Manual Mode Selector	I:1/5	Selects manual operation
Manual Run Push Button	I:1/6	Provides hold-to-run Manual request
Reset Push Button	I:1/7	Requests master-fault reset
Motor Feedback	I:1/8	Represents motor/contactor feedback
Motor Contactor	O:0/0	Motor starter/contactator output command
Running Light	O:0/1	Indicates commanded motor operation
Fault Light	O:0/2	Visible master-fault indication
Alarm Horn	O:0/3	Audible master-fault indication
Programming Software	RSLogix Micro Starter Lite 8.30	Ladder development, testing, and online monitoring

This project models motor-control logic in a PLC training/simulation environment. It does not represent a complete field installation with safety-rated emergency-stop hardware, contactor power circuitry, motor protection coordination, or machine commissioning.

5. I/O Assignment

Physical Inputs

Device	PLC Address	Type	Description
Start_PB	I:1/0	Digital Input	Automatic start push button
Stop_OK	I:1/1	Digital Input	Stop circuit healthy
EStop_OK	I:1/2	Digital Input	Emergency-stop circuit healthy
Overload_OK	I:1/3	Digital Input	Motor overload circuit healthy
Auto_Mode	I:1/4	Digital Input	Automatic mode selector
Manual_Mode	I:1/5	Digital Input	Manual mode selector
Manual_Run_PB	I:1/6	Digital Input	Manual hold-to-run command
Reset_PB	I:1/7	Digital Input	Master-fault reset command
Motor_Feedback	I:1/8	Digital Input	Motor/contactator feedback

Physical Outputs

Device	PLC Address	Type	Description
Motor_Contactor	O:0/0	Digital Output	Motor starter/contactor command
Running_Light	O:0/1	Digital Output	Commanded-running indication
Fault_Light	O:0/2	Digital Output	Master-fault indication
Alarm_Horn	O:0/3	Digital Output	Audible fault indication

Internal PLC Data

Function	Address	Type	Purpose
Start_Permissive	B3:0/0	Internal Bit	Master startup permission
Auto_Run_Request	B3:0/1	Internal Bit	Latched automatic run request
Mode_Run_Request	B3:0/2	Internal Bit	Non-latched Manual run request
Start_Request	B3:0/3	Internal Bit	Combined Auto/Manual start request
Fault_Active	B3:0/4	Internal Bit	Latched master fault
System_Ready	B3:0/5	Internal Bit	System ready status
Feedback_Fault	B3:0/7	Internal Bit	Live missing-feedback diagnostic
Run_Command	B3:1/0	Internal Bit	Final internal motor run command
Mode_Fault	B3:1/2	Internal Bit	Simultaneous Auto/Manual selection
One-Shot	B3:2/0	ONS Storage Bit	One-shot storage for C5:0 event
Runtime One-Shot	B3:2/1	ONS Storage Bit	One-shot storage for runtime interval
Start_Delay_Timer	T4:0	TON	Five-second startup delay
Feedback_Timer	T4:1	TON	Three-second feedback interval
Runtime_Timer	T4:2	TON	Ten-second simulation runtime interval
Runtime_Counter	C5:0	CTU	Counts completed startup-delay events; symbolic name preserved from final ladder
Runtime Accumulator	N7:0	Integer	Stores completed ten-second runtime intervals

6. Control Strategy

I organized the Version 2 program so operating validity is established before any request can reach the motor output. Auto_Mode and Manual_Mode are checked for simultaneous selection in Rung 0000. The resulting Mode_Fault, together with E-stop, overload, and feedback conditions, feeds the master Fault_Active latch in Rung 0001.

Rung 0003 centralizes startup permission. Start_Permissive requires healthy EStop_OK, Stop_OK, and Overload_OK inputs, no latched master fault, and exactly one selected operating mode. System_Ready then follows this permissive in Rung 0004.

The request architecture separates Auto and Manual behavior. Auto uses OTL/OTU logic: Start_PB latches Auto_Run_Request when Auto mode and System Ready are valid, while loss of Stop_OK or an active master fault unlatches the request. Manual mode instead uses Manual_Run_PB and an OTE Mode_Run_Request, so Manual operation is hold-to-run.

Rung 0008 combines the two request paths into Start_Request. Rung 0009 then requires both Start_Request and Start_Permissive before the five-second Start_Delay_Timer can complete. Run_Command is generated from the timer Done state when Fault_Active is false, and a dedicated output rung writes Motor_Contactor O:0/0.

Feedback monitoring is intentionally separate from command generation. Motor_Contactor starts Feedback_Timer T4:1. After the three-second interval has elapsed, the next rung evaluates Motor_Feedback I:1/8. If feedback is absent, Feedback_Fault becomes true and the master fault latch captures the abnormal condition. If feedback later disappears after the timer is already Done, the same rung also detects that loss while the motor command remains active.

The program records two types of operating information. C5:0 increments once on each rising event of T4:0/DN through ONS B3:2/0. Runtime_Timer T4:2 accumulates while Motor_Contactor is on; each ten-second Done event passes through ONS B3:2/1 and increments N7:0 by one before T4:2 is reset for the next interval.

7. Engineering Design Decisions

Design Decision	Reason
Separated requests from physical output	Allowed operating modes, permissives, timing, and faults to be evaluated before O:0/0 is energized
Used Mode_Fault B3:1/2	Explicitly identifies simultaneous Auto and Manual selection
Centralized Start_Permissive B3:0/0	Creates one clear decision point for startup permission
Created System_Ready B3:0/5	Provides an explicit readiness status for monitoring and troubleshooting
Latched Auto_Run_Request B3:0/1	Allows a momentary Start command to establish maintained Auto operation
Used OTE Mode_Run_Request B3:0/2	Keeps Manual operation dependent on the continued Manual Run command
Combined requests into Start_Request B3:0/3	Lets Auto and Manual share the same delay, command, output, feedback, and monitoring logic
Kept T4:0 at 5 seconds	Preserves the original Project 1 delayed-start function inside the V2 architecture
Used internal Run_Command B3:1/0	Separates final PLC command logic from the physical output

Used one Motor_Contactor output rung	Provides a single clear write point for O:0/0
Used T4:1 plus Motor_Feedback	Allows commanded operation to be checked against field/simulated response after a defined interval
Latched master Fault_Active	Retains abnormal conditions until active causes are restored and reset is accepted
Used ONS B3:2/0 before C5:0	Prevents the counter from incrementing on every scan while T4:0/DN remains true
Used T4:2 + ONS + ADD N7:0	Provides simple runtime accumulation that is easy to observe during simulation
Added Fault_Light and Alarm_Horn	Provides both visual and audible master-fault indication

One important implementation detail is that Auto_Run_Request is unlatched by Stop_OK loss or Fault_Active, but the final Rung 0006 does not include Auto-mode deselection as an unlatch condition. I preserve that implemented behavior rather than adding logic that is not present in the final ladder.

8. Ladder Logic Description

The final Project 1 Version 2 program contains functional Rungs 0000 through 0020, followed by END at Rung 0021. The typed representations below are reconstructed directly from the final RSLogix screenshots. Exact rung numbers, addresses, instruction types, symbolic names, branches, and presets are preserved.

8.1 Rung 0000 - Mode Fault Detection

Ladder Logic:

----[XIC I:1/4]----[XIC I:1/5]----- (OTE B3:1/2)----
Auto_Mode Manual_Mode Mode_Fault

Rung Description: When Auto_Mode I:1/4 and Manual_Mode I:1/5 are both true, Mode_Fault B3:1/2 energizes. Because the output is an OTE, Mode_Fault represents the current simultaneous-selection condition and turns off when the conflict disappears.

Engineering Purpose: I included this rung to make simultaneous Auto/Manual selection an explicit invalid operating state.

8.2 Rung 0001 - Master Fault Latch

Ladder Logic:

-----+-----[XIO I:1/2]-----+----- (OTL B3:0/4)----
EStop_OK Fault_Active
+-----[XIO I:1/3]-----+
Overload_OK
+-----[XIC B3:1/2]-----+
Mode_Fault
+-----[XIC B3:0/7]-----+
Feedback_Fault

Rung Description: Any unhealthy E-stop condition, unhealthy overload condition, active Mode_Fault, or active Feedback_Fault can latch Fault_Active B3:0/4. The OTL stores the master fault after the initiating source disappears.

Engineering Purpose: The latched master fault creates one common fault state for shutdown, indication, and deliberate recovery.

8.3 Rung 0002 - Master Fault Reset

Ladder Logic:

```
--[ XIC I:1/7 ]--[ XIC I:1/2 ]--[ XIC I:1/3 ]---[ XIO B3:1/2 ]---[ XIO B3:0/7 ]---( OTU B3:0/4 )--[
Reset_PB      EStop_OK      Overload_OK      Mode_Fault      Feedback_Fault      Fault_Active
```

Rung Description: Reset_PB can unlatch Fault_Active only when EStop_OK and Overload_OK are healthy and both Mode_Fault and Feedback_Fault are false. Stop_OK is not part of this reset rung.

Engineering Purpose: I used conditional reset logic so the master fault cannot be cleared while one of the monitored fault sources is still active.

8.4 Rung 0003 - Start Permissive

Ladder Logic:

[illegible]

Rung Description: Start_Permissive B3:0/0 requires healthy E-stop, Stop, and overload inputs, no latched master fault, and exactly one selected mode. The parallel mode branch permits either Auto true/Manual false or Manual true/Auto false.

Engineering Purpose: This rung centralizes the complete startup-permission decision and makes readiness easier to troubleshoot.

8.5 Rung 0004 - System Ready

Ladder Logic:

```
|---[ XIC B3:0/0 ]----- ( OTE B3:0/5 )---|
      Start Permissive                System Ready
```

Rung Description: System_Ready B3:0/5 directly follows Start_Permissive. The mode validation does not need to be repeated here because it is already included in Rung 0003.

Engineering Purpose: I separated readiness indication from the detailed permissive logic so the overall machine-ready state is easy to monitor.

8.6 Rung 0005 - Automatic Run Request Latch

Ladder Logic:

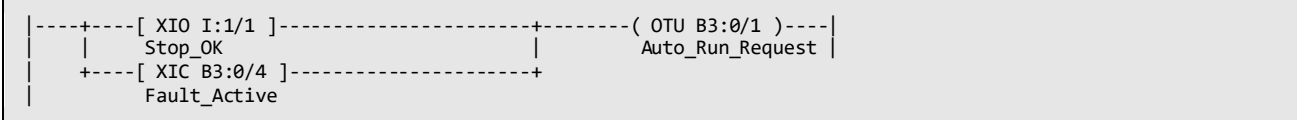
```
|----[ XIC I:1/0 ]----[ XIC I:1/4 ]----[ XIC B3:0/5 ]----- ( OTL B3:0/1 )----|
      Start PB           Auto Mode           System Ready           Auto Run Request
```

Rung Description: A valid Start_PB command in Auto mode while System_Ready is true latches Auto_Run_Request B3:0/1. The request remains stored after Start_PB is released.

Engineering Purpose: The latched request lets the automatic sequence continue from a momentary operator command without directly latching the motor output.

8.7 Rung 0006 - Automatic Run Request Unlatch

Ladder Logic:

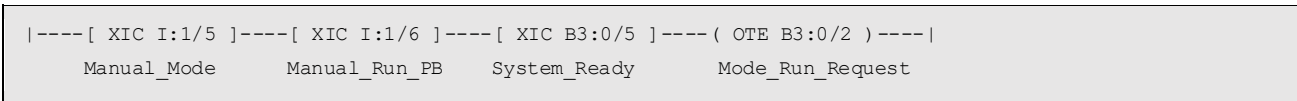


Rung Description: Auto_Run_Request is unlatched when Stop_OK becomes false or Fault_Active becomes true. The final rung does not include Auto-mode deselection as an unlatch condition.

Engineering Purpose: This rung removes the maintained Auto request following normal Stop or a master fault so a new Start command is required after those events.

8.8 Rung 0007 - Manual Mode Run Request

Ladder Logic:

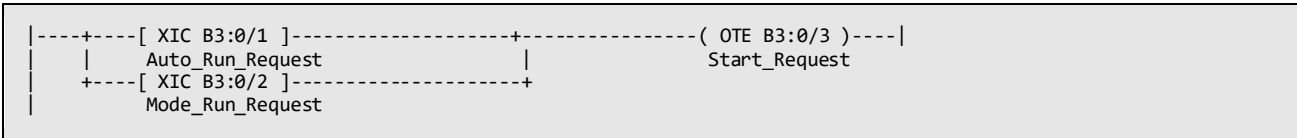


Rung Description: Mode_Run_Request B3:0/2 is true only while Manual mode, Manual_Run_PB, and System_Ready are all true. Because the output is an OTE, releasing Manual_Run_PB immediately removes the request.

Engineering Purpose: I used non-latched Manual control to create hold-to-run behavior for the Manual operating path.

8.9 Rung 0008 - Combined Start Request

Ladder Logic:

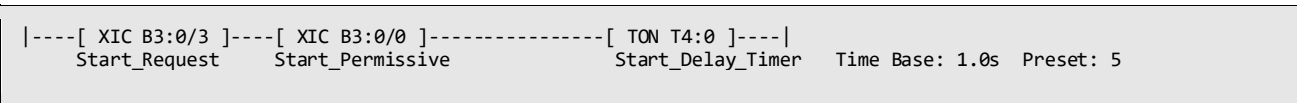


Rung Description: Start_Request B3:0/3 becomes true when either Auto_Run_Request or Mode_Run_Request is active.

Engineering Purpose: Combining the requests lets both operating modes share the same downstream timing, command, output, feedback, and monitoring logic.

8.10 Rung 0009 - Motor Startup Delay

Ladder Logic:



Rung Description: T4:0 accumulates only while both Start_Request and Start_Permissive are true. After five seconds, T4:0/DN becomes true. Because TON is non-retentive, loss of the enabling path resets the timer.

Engineering Purpose: This rung preserves the original Project 1 start-delay requirement while placing it inside the Version 2 request/permissive structure.

8.11 Rung 0010 - Run Command

Ladder Logic:

```
|----[ XIC T4:0/DN ]----[ XIO B3:0/4 ]----- ( OTE B3:1/0 )----|
      Start_Delay_Timer      Fault_Active              Run_Command
```

Rung Description: Run_Command B3:1/0 becomes true when the startup-delay Done bit is true and Fault_Active is false. An active master fault immediately removes Run_Command.

Engineering Purpose: I used an internal Run_Command to separate the final control decision from the physical Motor_Contactor output.

8.12 Rung 0011 - Motor Contactor Output

Ladder Logic:

```
|----[ XIC B3:1/0 ]----- ( OTE O:0/0 )----|
      Run_Command              Motor_Contactor
```

Rung Description: Motor_Contactor O:0/0 follows Run_Command B3:1/0. This is the single rung that writes the physical motor output.

Engineering Purpose: A dedicated output rung provides one clear point of control and simplifies troubleshooting.

8.13 Rung 0012 - Running Light

Ladder Logic:

```
|----[ XIC O:0/0 ]----- ( OTE O:0/1 )----|
      Motor_Contactor              Running_Light
```

Rung Description: Running_Light O:0/1 follows the PLC's Motor_Contactor command. It indicates commanded operation, not verified motor feedback.

Engineering Purpose: I included a separate running indication so the commanded state is visible without monitoring the internal Run_Command.

8.14 Rung 0013 - Feedback Timer

Ladder Logic:

```
|----[ XIC O:0/0 ]-----[ TON T4:1 ]----|
      Motor_Contactor              Feedback_Timer  Time Base: 1.0 s  Preset: 3
```

Rung Description: Feedback_Timer T4:1 runs whenever Motor_Contactor is energized. Motor_Feedback is not in this timer rung, so the timer reaches Done after three seconds as long as the contactor command remains on.

Engineering Purpose: The timer establishes a defined validation interval before the next rung evaluates whether expected feedback is present.

8.15 Rung 0014 - Feedback Fault

Ladder Logic:

```
|----[ XIC T4:1/DN ]----[ XIO I:1/8 ]----- ( OTE B3:0/7 )----|
      Feedback_Timer      Motor_Feedback          Feedback_Fault
```

Rung Description: After T4:1 reaches Done, Feedback_Fault B3:0/7 becomes true whenever Motor_Feedback I:1/8 is false. Because the output is an OTE, Feedback_Fault is a live diagnostic condition rather than a latched bit. Fault_Active is the latched master response.

Engineering Purpose: This rung verifies that commanded motor operation is accompanied by the expected feedback after the allowed response interval.

8.16 Rung 0015 - Completed Start Event Counter

Ladder Logic:

```
|----[ XIC T4:0/DN ]----[ ONS B3:2/0 ]-----[ CTU C5:0 ]----|
      Start_Delay_Timer      One-Shot          Runtime_Counter  PRE: 9999
```

Rung Description: When T4:0/DN transitions false-to-true, ONS B3:2/0 allows CTU C5:0 to increment once. The final screenshot labels C5:0 as Runtime_Counter; that symbolic name is preserved here even though the enabling event is completion of the startup delay.

Engineering Purpose: The one-shot prevents repeated counts on every PLC scan while T4:0/DN remains true.

8.17 Rung 0016 - Runtime Timer

Ladder Logic:

```
|----[ XIC 0:0/0 ]-----[ TON T4:2 ]----|
      Motor_Contactor          Runtime_Timer  Time Base: 1.0 s  Preset: 10
```

Rung Description: Runtime_Timer T4:2 accumulates whenever Motor_Contactor is on. The ten-second preset is used for simulation so repeated runtime intervals can be observed quickly.

Engineering Purpose: The short interval provides a practical way to demonstrate runtime accumulation during PLC testing.

8.18 Rung 0017 - Runtime Accumulation

Ladder Logic:

```
|----[ XIC T4:2/DN ]----[ ONS B3:2/1 ]-----[ ADD ]----|
      Runtime_Timer      B3:2/1          Source A: N7:0
                                   Source B: 1
                                   Dest:    N7:0
```

Rung Description: When T4:2/DN becomes true, ONS B3:2/1 creates a one-scan pulse and the ADD instruction increments N7:0 by one.

Engineering Purpose: I used an integer accumulator for completed runtime intervals. This is the final implementation and replaces the earlier C5:1 concept described in supporting documents.

8.19 Rung 0018 - Runtime Timer Reset

Ladder Logic:

```
|----[ XIC T4:2/DN ]-----[ RES T4:2 ]----|
Runtime_Timer                Runtime_Timer
```

Rung Description: When T4:2/DN is true, Rung 0018 resets T4:2. Because Rung 0017 is scanned first, the ADD event is processed before the timer is reset. If Motor_Contactor remains on, timing begins again on the next interval.

Engineering Purpose: This creates repeating ten-second runtime intervals while the motor remains commanded on.

8.20 Rung 0019 - Fault Light

Ladder Logic:

```
|----[ XIC B3:0/4 ]----- ( OTE O:0/2 )----|
Fault_Active                Fault_Light
```

Rung Description: Fault_Light O:0/2 remains on while the latched Fault_Active bit is true.

Engineering Purpose: The dedicated visual indication distinguishes a faulted stop from normal stopped operation.

8.21 Rung 0020 - Alarm Horn

Ladder Logic:

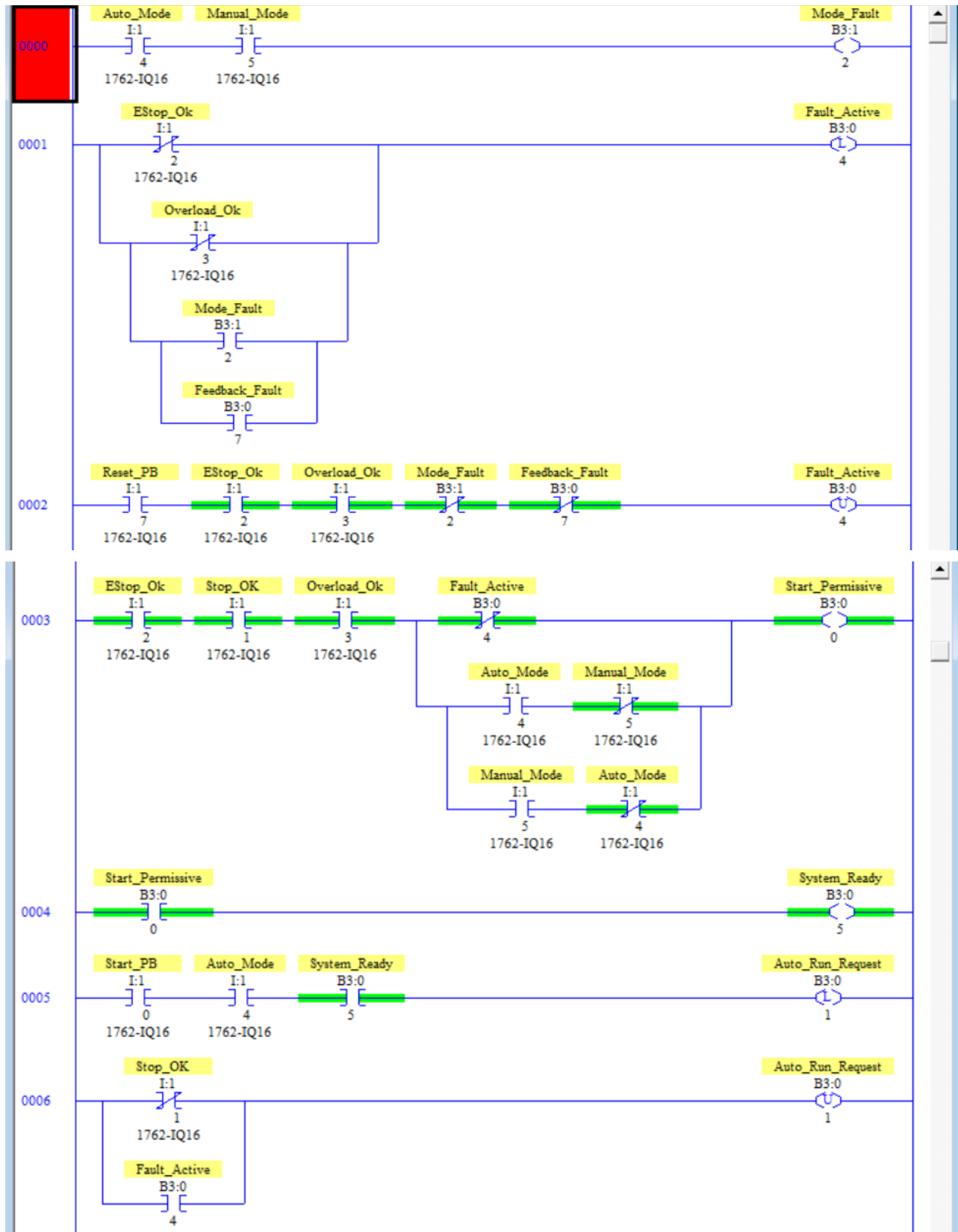
```
|----[ XIC B3:0/4 ]----- ( OTE O:0/3 )----|
Fault_Active                Alarm_Horn
```

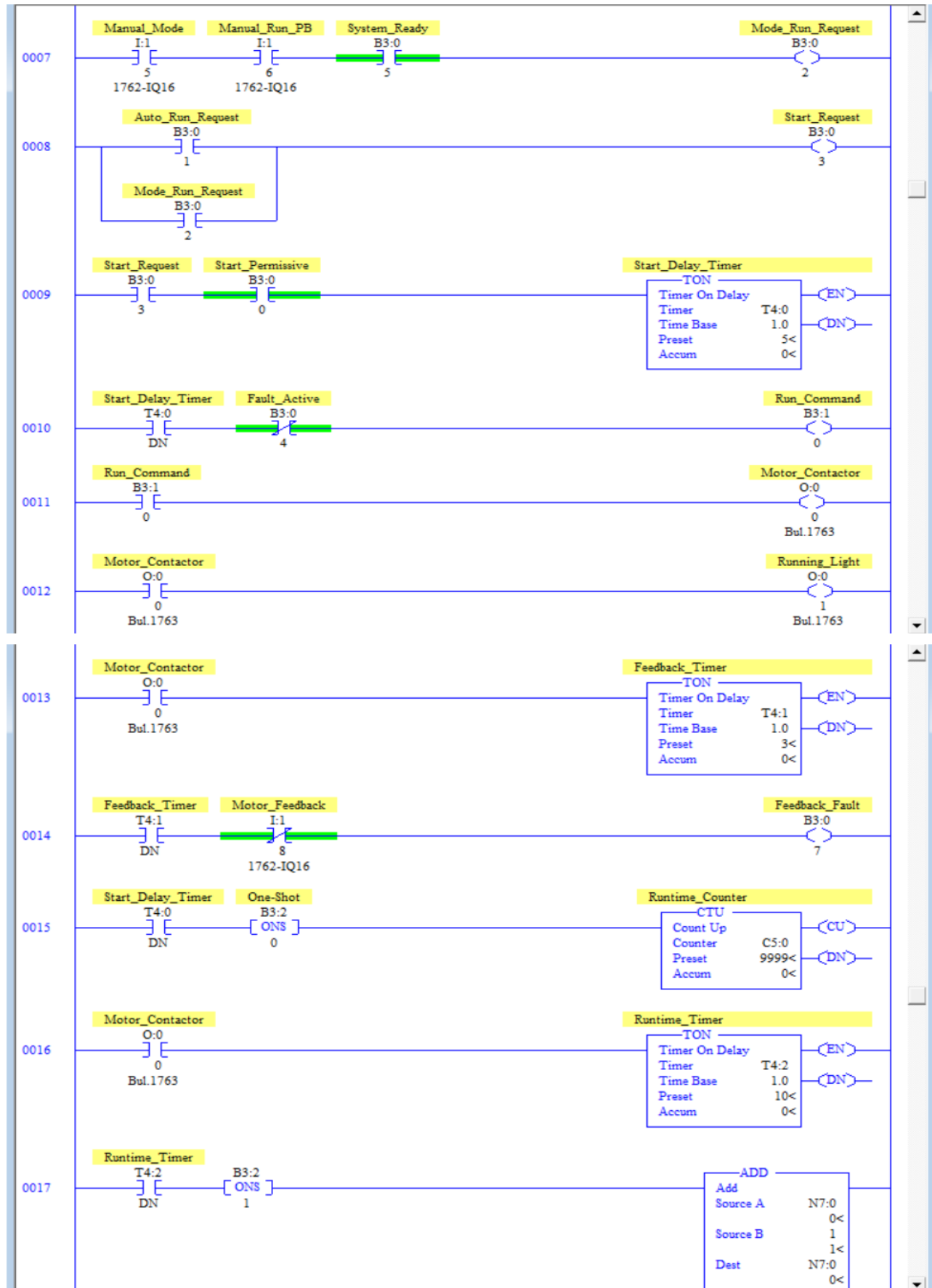
Rung Description: Alarm_Horn O:0/3 also follows Fault_Active and remains energized until the master fault is cleared.

Engineering Purpose: The audible alarm attracts attention to a fault condition even when the operator is not directly observing the fault light.

8.22 Actual RSLogix Implementation

The figures below show the final RSLogix ladder used as the technical source of truth for this report.





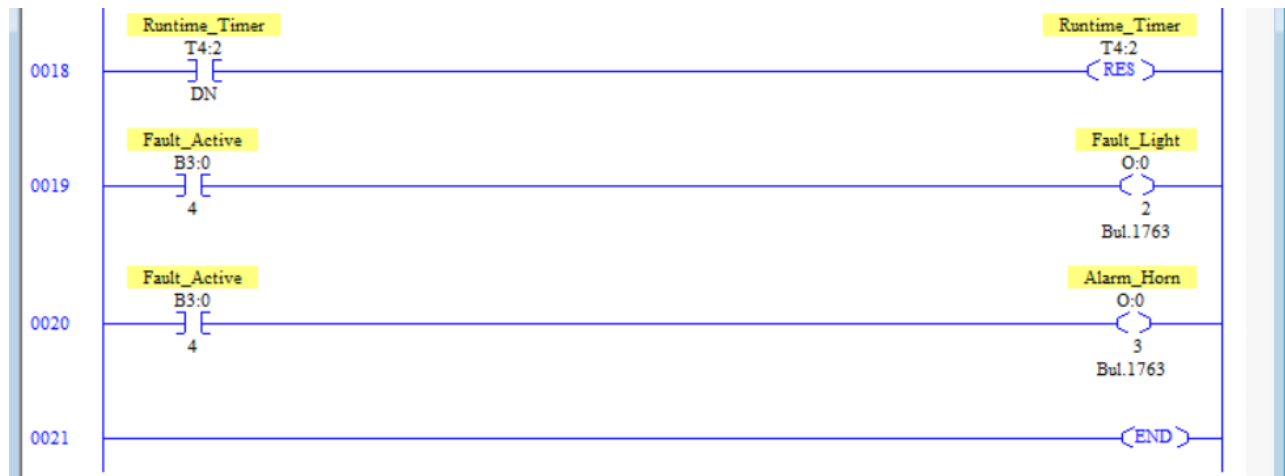


Figure P1V2.1 - Final RSLogix Implementation

9. Sequence of Operation

Step 1 - Initial Condition: The PLC continuously evaluates the protective inputs, master fault state, and mode selection. Motor_Contactor remains off until the complete request, permissive, delay, and Run Command path is established.

Step 2 - Select Operating Mode: The operator selects either Auto_Mode or Manual_Mode. Selecting both simultaneously energizes Mode_Fault.

Step 3 - Establish Start Permissive: With EStop_OK, Stop_OK, and Overload_OK healthy, Fault_Active clear, and exactly one valid mode selected, Start_Permissive turns on.

Step 4 - System Ready: System_Ready follows Start_Permissive and indicates that the implemented startup conditions are satisfied.

Step 5 - Auto Request: In Auto mode, pressing Start_PB while System_Ready is on latches Auto_Run_Request.

Step 6 - Manual Request: In Manual mode, holding Manual_Run_PB while System_Ready is on energizes Mode_Run_Request. Releasing Manual_Run_PB removes the request.

Step 7 - Combined Start Request: Either operating request energizes Start_Request.

Step 8 - Five-Second Delay: Start_Request and Start_Permissive enable T4:0. Motor_Contactor remains off while T4:0 accumulates to five seconds.

Step 9 - Run Command: When T4:0/DN is true and Fault_Active is false, Run_Command energizes.

Step 10 - Motor Command: Run_Command energizes Motor_Contactor O:0/0, and Running_Light O:0/1 follows the motor command.

Step 11 - Feedback Validation: Motor_Contactor starts Feedback_Timer T4:1. After three seconds, the PLC checks Motor_Feedback. Missing feedback makes Feedback_Fault true.

Step 12 - Master Fault Response: An active Feedback_Fault, Mode_Fault, unhealthy EStop_OK, or unhealthy Overload_OK can latch Fault_Active. Fault_Active removes Run_Command and energizes Fault_Light and Alarm_Horn.

Step 13 - Start Event Counting: When T4:0/DN first becomes true, ONS B3:2/0 allows C5:0 to increment once.

Step 14 - Runtime Accumulation: While Motor_Contactor is on, T4:2 measures ten-second intervals. Each completed interval increments N7:0 once through ONS B3:2/1 and ADD.

Step 15 - Runtime Timer Reset: T4:2 is reset after each completed interval so the next ten-second period can begin.

Step 16 - Normal Auto Stop: If Stop_OK becomes false, Auto_Run_Request is unlatched, the request path is removed, T4:0 resets, and the motor command turns off.

Step 17 - Fault Recovery: After the active fault source is restored, Reset_PB can unlatch Fault_Active when the reset rung conditions are satisfied. Start_Permissive and System_Ready then return when all required conditions are again valid.

10. Testing & Validation

I tested the final ladder version in the PLC development/simulation environment. The final ladder screenshots are the technical source of truth for the tested implementation, and all defined Project 1 Version 2 requirements passed. The test descriptions below have therefore been aligned to the final addresses and logic rather than the older supporting-document address map.

Test ID	Test Action	Expected Result	Observed Result	Status
T-01	Set EStop_OK, Stop_OK, Overload_OK healthy; select Auto only	Start_Permissive and System_Ready turn ON; motor remains OFF	Ready-state logic operated as expected	PASS
T-02	Press and release Start_PB in Auto mode	Auto_Run_Request latches; T4:0 begins timing	Automatic request remained latched and delay started	PASS
T-03	Observe startup delay	Motor remains OFF for 5 s; T4:0/DN then turns ON	Five-second delay completed correctly	PASS
T-04	Allow valid Auto startup to complete	Run_Command, Motor_Contactor, and Running_Light turn ON	Motor command and running indication energized	PASS
T-05	Turn Motor_Feedback ON before/after feedback interval as required for normal operation	Feedback_Fault remains OFF while feedback is present	Normal feedback condition maintained operation	PASS
T-06	Turn Stop_OK OFF during Auto run	Auto_Run_Request unlatches; motor command turns OFF; no restart from restoring Stop_OK alone	Motor stopped and stored Auto request cleared	PASS
T-07	Select Manual mode and hold Manual_Run_PB	Mode_Run_Request follows button; common 5 s startup path is used	Manual hold-to-run behavior operated correctly	PASS
T-08	Select Auto and Manual together	Mode_Fault turns ON; Fault_Active latches; Start_Permissive is blocked	Mode conflict produced the expected fault response	PASS
T-09	Turn EStop_OK OFF	Fault_Active latches; motor command is removed; fault outputs turn ON	E-stop monitoring produced the expected response	PASS
T-10	Turn Overload_OK OFF	Fault_Active latches; motor command is removed; fault outputs turn ON	Overload monitoring produced the expected response	PASS
T-11	Keep Motor_Feedback OFF after T4:1 reaches 3 s	Feedback_Fault turns ON; Fault_Active latches; motor command drops	Missing-feedback fault operated as expected	PASS
T-12	Restore fault source and press Reset_PB with reset conditions valid	Fault_Active unlatches; fault outputs turn OFF; system can return Ready	Fault reset and recovery operated correctly	PASS

T-13	Perform repeated completed startup-delay events	ONS B3:2/0 allows C5:0 to increment once per event	C5:0 increased once per completed start-delay event	PASS
T-14	Run motor through repeated 10 s T4:2 intervals	Each interval increments N7:0 once and T4:2 resets	N7:0 runtime accumulation repeated correctly	PASS

Validation Summary

Testing confirmed the final Version 2 behavior for readiness, Auto operation, Manual hold-to-run operation, five-second startup delay, normal stop, mode conflict, E-stop monitoring, overload monitoring, feedback failure, master-fault reset, completed-start event counting, and N7:0 runtime accumulation. All defined requirements passed on the final ladder version.

The testing represents functional PLC program validation in the project development/simulation environment. It does not represent physical machine commissioning, field acceptance testing, or safety-system validation.

11. Results

The completed Project 1 Version 2 program performed the required motor-control, mode-management, fault, feedback, indication, counting, and runtime functions in the final tested ladder version.

The program accepted Auto and Manual requests through separate control paths and required the centralized Start_Permissive before startup timing could proceed. The five-second Start_Delay_Timer prevented Motor_Contactor from energizing until the configured delay was complete.

The final architecture separated the operating request from Run_Command and Motor_Contactor, making the control path easier to trace. Feedback_Timer T4:1 and Motor_Feedback I:1/8 provided a defined feedback-verification function, while Fault_Active retained abnormal conditions for deliberate recovery.

C5:0 recorded completed startup-delay events once through ONS B3:2/0. Runtime accumulation used T4:2, ONS B3:2/1, ADD, and N7:0. The ten-second simulation interval repeatedly incremented N7:0 while the motor command remained active.

All defined final-version functional tests passed.

12. Challenges & Troubleshooting

A useful troubleshooting path for this project is: Protective Inputs / Mode Selection -> Mode_Fault / Fault_Active -> Start_Permissive -> System_Ready -> Operating Request -> Start_Request -> T4:0 -> Run_Command -> Motor_Contactor -> Feedback Timer / Feedback -> Fault Response.

For runtime problems, I continue the trace through: Motor_Contactor -> T4:2 -> T4:2/DN -> ONS B3:2/1 -> ADD -> N7:0 -> RES T4:2.

Symptom	Possible Cause	Diagnostic Check	Corrective Action
System Ready does not turn ON	Unhealthy protective input, active fault, or invalid mode selection	Monitor I:1/1, I:1/2, I:1/3, B3:0/4, I:1/4, I:1/5	Restore healthy conditions and select exactly one mode
Auto request does not latch	Start_PB, Auto_Mode, or System_Ready missing	Trace Rung 0005	Verify I:1/0, I:1/4, and B3:0/5

Auto request does not clear on Stop	Stop_OK did not go false or OTU path is not true	Monitor I:1/1 and B3:0/1	Verify Rung 0006 and Stop input behavior
Manual request does not stay ON	Manual_Run_PB released or System_Ready lost	Monitor I:1/5, I:1/6, B3:0/5, B3:0/2	Verify Manual mode and hold-to-run command
Startup timer does not count	Start_Request or Start_Permissive false	Monitor B3:0/3, B3:0/0, T4:0/EN	Trace Rungs 0008-0009
Timer completes but motor does not run	Fault_Active true or Run_Command path false	Monitor T4:0/DN, B3:0/4, B3:1/0, O:0/0	Trace Rungs 0010-0011
Motor faults after startup	Motor_Feedback absent after T4:1 done	Monitor T4:1/DN, I:1/8, B3:0/7	Verify simulated/field feedback behavior
Fault does not reset	EStop_OK/Overload_OK unhealthy or Mode_Fault/Feedback_Fault still active	Trace Rung 0002	Restore fault source before Reset
C5:0 increments incorrectly	ONS event behavior incorrect	Monitor T4:0/DN, B3:2/0, C5:0.ACC	Verify Rung 0015
N7:0 does not increment	T4:2 not reaching Done or ADD event not executing	Monitor O:0/0, T4:2/DN, B3:2/1, N7:0	Trace Rungs 0016-0018
Fault light/horn remain ON	Fault_Active remains latched	Monitor B3:0/4 and fault-source rungs	Correct active cause and complete valid reset

One important troubleshooting lesson was to trace the control path from the left side of the program rather than beginning at the motor output. A motor that does not start may have a correct output rung but never receive Run_Command because the problem occurred earlier in mode selection, permissive logic, the operating request, or startup timing.

13. Lessons Learned

This project helped me understand how a simple delayed-start requirement can be separated into distinct control functions instead of being implemented as one direct path from a push button to an output.

I learned the value of separating operator intention from equipment command. Auto_Run_Request and Mode_Run_Request represent the requested operating state, while Start_Request, T4:0, Run_Command, and Motor_Contactor represent later stages of the decision process.

The project strengthened my understanding of centralized permissive logic. Start_Permissive gives me one defined place to determine why startup is or is not allowed instead of repeating the same conditions in multiple output rungs.

I also learned why Auto and Manual modes can use different request behavior. Auto uses a maintained request through OTL/OTU logic, while Manual uses an OTE and a dedicated Manual_Run_PB for hold-to-run control.

Feedback monitoring reinforced the difference between commanding equipment and verifying response. Running_Light follows the PLC command, while Motor_Feedback is evaluated separately after a defined interval.

The one-shot instructions reinforced scan-based event handling. They allow a persistent timer Done condition to create only one count or one runtime increment per completed interval.

Finally, this project reinforced the importance of source-of-truth discipline. During documentation I found that earlier supporting documents contained older addresses and an earlier runtime concept. Using the final tested ladder as the controlling reference prevented those obsolete details from entering the portfolio.

14. Industrial Relevance

Motor control with permissives, operating modes, fault handling, command/feedback separation, and runtime tracking is relevant to manufacturing equipment, conveyors, pumps, fans, mixers, packaging machines, and other automated systems.

This Version 2 project demonstrates how a basic motor function can be organized into clearer control responsibilities: operating-mode validation, readiness, maintained versus momentary requests, delayed startup, command generation, feedback diagnostics, master-fault response, operator indication, and equipment-use information.

The project remains a PLC training and portfolio implementation rather than a production machine installation. In an actual industrial system, emergency-stop and personnel-safety functions would require appropriately engineered safety-rated devices, field wiring, and validation. The standard PLC logic here monitors the E-stop healthy status and uses it in the control response; it is not presented as the sole personnel-protection layer.

15. Version 1 to Version 2 Engineering Progression

Version 1 Foundation	Version 2 Implementation	Engineering Benefit
Basic motor start-delay function	T4:0 retained inside request/permissive architecture	Preserves the original objective while improving control structure
Single operating path	Separate Auto and Manual paths	Supports maintained Auto and hold-to-run Manual behavior
No explicit mode-conflict handling	Mode_Fault B3:1/2	Defines simultaneous mode selection as invalid
Limited run-enabling logic	Centralized Start_Permissive B3:0/0	Provides one clear startup-permission decision
No explicit ready status	System_Ready B3:0/5	Improves monitoring and troubleshooting
Operator command close to motor output	Request -> Start_Request -> T4:0 -> Run_Command -> Motor_Contactor	Separates intention, timing, decision, and physical output
No master fault architecture	Latched Fault_Active B3:0/4	Retains abnormal conditions for deliberate recovery
No motor response verification	T4:1 + Motor_Feedback + Feedback_Fault	Compares commanded operation with expected feedback
No start-event record	ONS B3:2/0 + CTU C5:0	Counts completed startup-delay events once
No runtime accumulation	T4:2 + ONS B3:2/1 + ADD N7:0	Tracks repeated simulated operating intervals
Limited operator indication	Running_Light, Fault_Light, Alarm_Horn	Provides clearer operating and fault visibility

The main improvement is not simply that Version 2 contains more rungs. The engineering progression is the separation of responsibilities. Version 1 established a delayed motor start; Version 2 determines whether operation is permitted, which mode requested it, whether the delay completed, whether a master fault exists, whether the motor responded, and how operating events are recorded.

16. Project Summary

Project 1 Version 2 upgraded the original motor start-delay application into a structured PLC motor-control project using an Allen-Bradley MicroLogix 1100 and RSLogix Micro Starter Lite 8.30.

The final ladder includes Auto and Manual operating modes, simultaneous-mode detection, centralized Start_Permissive logic, System_Ready status, separate operating requests, a five-second startup delay, an internal Run_Command, a dedicated Motor_Contactor output, motor feedback monitoring, a latched master fault, visual and audible fault indication, completed-start event counting, and runtime accumulation.

A key design improvement was separating the operator request from the physical output. The resulting control path progresses through mode and permissive validation before a request reaches the startup timer and Run_Command.

Motor response is checked with Feedback_Timer T4:1 and Motor_Feedback I:1/8. Missing feedback after the three-second interval makes Feedback_Fault active and causes the master Fault_Active condition to latch.

The final program records completed startup-delay events in C5:0 through ONS B3:2/0 and accumulates ten-second runtime intervals in N7:0 through T4:2, ONS B3:2/1, ADD, and RES logic.

All defined final-version requirements passed during PLC simulation/testing. The project demonstrates my progression from implementing a basic timer-controlled motor function to organizing the same requirement around operating modes, permissives, internal commands, feedback, diagnostics, recovery, and equipment-use information while remaining within the scope of a training and portfolio implementation.